ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

on

DevDocs: Software Architecture Patterns

By

Ayush Mishra

150096725046

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Technology Applied
5. Implementation Details and Snapshots
6. Results and Conclusion
7. References

1. Introduction to the Case Study

DevDocs is a web-based portal made for software architects and engineers. It gives them easy access to system design blueprints, architecture patterns, and performance metrics. The project is built using only HTML5, CSS3, and JavaScript — no backend or external library is used.

Many software developers find it hard to learn architecture patterns because the information is spread across different books, websites, and videos. DevDocs brings everything into one clean and organised place.

The portal has a dark and light mode interface, a sidebar for navigation, a real-time search feature, and a subscription pricing section. It is designed to look and feel like a real professional SaaS product.

2. Problem Statement / Case Background (Abstract)

Background

Software developers and architects often need to choose the right architecture for their system. But they have to search across many different sources — books, blogs, and videos — to understand trade-offs, cost, and scale. There is no single clean reference tool available.

Abstract

DevDocs solves this problem by providing a subscription-gated portal where 16 architecture patterns are shown in a clear card format. Each card shows the technology stack, pros and cons, expected user scale, latency type, and estimated monthly infrastructure cost.

The project is built using:

- HTML5 for structure and layout
- CSS3 for styling, dark/light mode, and animations
- Vanilla JavaScript for data rendering, search filtering, and theme toggle

The goal is to help developers make fast and confident architecture decisions by giving them all the information they need in one place.

3. Problem Statement / Case Study Design

3.1 Overall Layout

The application uses a Sidebar + Main Content layout, similar to a professional SaaS dashboard. It has the following main parts:

- Sidebar (260px fixed): Has the DevDocs logo, three navigation links (Patterns, Pricing, Performance), a theme toggle button, and a user profile card at the bottom.
- Main Content Area: Switches between three sections — Patterns, Pricing, and Performance — using JavaScript.
- Landing Page: A full-screen intro screen shown when the website first loads. It fades out when the user clicks 'Enter Portal'.

3.2 Page Flow

The flow of the application is as follows:

```
LANDING PAGE  →  Click 'Enter Portal'  →  MAIN PORTAL
MAIN PORTAL  →  Sidebar: [Patterns] [Pricing] [Performance]
Patterns: 16 Cards + Search Filter
Pricing: 3 Plans (Starter / Pro / Enterprise)
Performance: 15 KPI Metric Cards
```

3.3 Visual Design

- Font: Outfit (for all text) and JetBrains Mono (for code and metrics)
- Dark Mode Colors: Background #080b0f, Surface #0f1318, Green Accent #00ffa3
- Light Mode Colors: Background #f0f4f8, Surface #ffffff, same accents
- Cards: 14px border radius, hover effect lifts the card up with a green glow
- Animations: Smooth fade-up on page load, slide-in sidebar transition

4. Methods & Technology Applied

4.1 HTML5

- Semantic layout using aside, nav, section, div, and button elements
- Architecture diagrams shown using styled div blocks with monospace text
- Code snippets shown using pre tags with show/hide toggle button
- Tech stack shown as badge-style span elements
- Pros and cons listed inside each pattern card

4.2 CSS3

- CSS Variables used for dark and light mode theming (--bg, --surface, --acc)
- Grid layout: repeat(auto-fill, minmax(300px, 1fr)) for responsive cards
- Sidebar hidden/shown using transform: translateX(-100%) with smooth transition
- Hover effects: cards lift up with translateY(-4px) and show a border glow
- Animations: fadeUp keyframe animation on cards when page loads
- Light mode uses [data-theme='light'] to override all CSS variables

4.3 JavaScript

- All 16 patterns stored in a patterns array in script.js
- filterPatterns(query) function searches across name, scale, latency, tech, tag, pros, cons
- renderHome() function renders pattern cards dynamically using template literals
- renderPricing() renders 3 pricing plan cards
- renderPerf() renders 15 performance metric cards
- toggleTheme() switches between dark and light mode and saves to localStorage
- toggleSidebar() hides or shows the sidebar and shifts the main content
- highlight(text, query) wraps matched search text in mark tags for yellow highlight
- escHtml(str) safely renders code inside pre tags without breaking HTML

Technology	Used For	Detail
HTML5	Structure	Semantic layout, cards, forms
CSS3	Styling	Variables, Grid, Animations
JavaScript	Functionality	Data, Search, Theme, Render
Google Fonts	Typography	Outfit + JetBrains Mono
localStorage	Persistence	Saves dark/light preference

5. Implementation Details

5.1 File Structure

File	Purpose
index.html	Main portal — Landing screen, Sidebar, Patterns, Pricing, Performance sections
patterns.html	Detailed patterns page with architecture diagrams and code snippet toggle
search.html	Search page with large input, filter chips, and highlighted results
style.css	All styling — dark/light mode, cards, sidebar, animations, pricing
script.js	All data (16 patterns, pricing, metrics) and all helper functions

5.2 Key Features Implemented

Feature	How It Works
Landing Screen	Full-screen overlay that fades out and is removed from DOM on click
Sidebar Toggle	sidebar.classList.toggle('hidden') slides it off screen, content expands
Live Search	input event filters patterns array on every keystroke
Filter Chips	search.html has 7 chips (Low Latency, Cloud, etc.) that pre-fill the search
Code Snippet Toggle	Show/Hide button toggles display:block on pre element
Dark/Light Mode	data-theme attribute on html tag switches all CSS variables instantly
Most Popular Badge	Badge is opacity:0 by default, shows on hover via CSS
Pricing Highlight	All pricing cards glow green on hover, Pro card glows blue

5.3 16 Architecture Patterns

#	Pattern	Scale	Latency	Cost	Tech Stack
1	Monolith	1K users	Low	₹1,260	Node.js, PostgreSQL
2	MVC Pattern	5K users	Low	₹2,100	Ruby on Rails
3	Layered Arch	10K users	Medium	₹3,360	Java, Spring Boot
4	Hexagonal	50K users	Low	₹5,040	Python, FastAPI
5	Micro-kernel	80K users	Low	₹10,080	C++, Qt
6	Serverless FaaS	200K users	Variable	₹21,000	AWS Lambda
7	CQRS	400K users	Low	₹37,800	Kafka, .NET
8	Event Sourcing	600K users	Medium	₹46,200	EventStoreDB
9	Pub-Sub	800K users	Low	₹58,800	RabbitMQ, Redis
10	Microservices	1M users	Medium	₹1.01L	Go, Kubernetes
11	API Gateway	5M users	Low	₹1.68L	Kong, Nginx
12	Saga Pattern	10M users	High	₹2.94L	Java, Temporal
13	DB Sharding	25M users	Medium	₹4.20L	Vitess, MySQL
14	Cell-Based Arch	50M users	Medium	₹10.08L	AWS, Terraform
15	Sidecar Pattern	50M+ users	Low	₹12.60L	Istio, Envoy
16	Bulkhead	100M users	High	₹15.12L	Hystrix, Resilience4j

6. Results and Conclusion

Results

The DevDocs portal was successfully built and works correctly in all modern browsers. Below is a summary of what was achieved:

- All 16 architecture pattern cards are displayed with full details — tech stack, pros, cons, scale, latency, and cost.
- The live search filter works correctly — typing '1M users' shows only the Microservices card.
- The 'Show Code' toggle button correctly shows and hides code snippets on patterns.html.
- The 7 filter chips on search.html correctly filter patterns by category.
- Dark and light mode toggle works and the choice is saved using localStorage.
- The sidebar hides and shows smoothly with the toggle button.
- The 'Most Popular' badge on the Pro pricing card correctly appears only on hover.
- The landing screen fades out smoothly and is removed from the page after clicking 'Enter Portal'.

Conclusion

DevDocs successfully demonstrates how a professional, production-quality web portal can be built using only HTML5, CSS3, and Vanilla JavaScript — without any backend or external library.

The project covers all required topics from Case Study No. 50:

- Clean HTML structure with semantic elements
- CSS animations, responsive layout, and full theme switching
- JavaScript data arrays, dynamic rendering, live search, and localStorage

This project shows strong understanding of front-end web development, including DOM manipulation, event handling, CSS animations, responsive design, and state persistence.

7. References

1. ITM Skills University — Case Study No. 50 Problem Statement
2. W3 school